Project - Low Level Design

on

(sports content generator)

Course Name: Generative - AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	ABHAY SINGH THAKUR	EN22CS304003
2	SARTHAK GOKHALE	EN22CS304057
3	HIMANSHI PORWAL	EN22CS306021
4	AKASH MUVEL	EN23EE3L1001
5	PRATHAM JOSHI	EN22CS306037

Group Name: ***Group 04D11***

*Project Number:****DO-30***

Industry Mentor Name: ***Mr. Yash***

*University Mentor Name:****Dr Hemlata patel***

*Academic Year:****2022-2026***

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

1.1 Scope of the document

This Low Level Design (LLD) document describes the internal design of the Sports_GenAi project. It explains the frontend and backend structure, the content generation workflow, the AI integration, the request and response format, and the major design decisions used in the implementation.

1.2 Intended Audience

This document is intended for faculty evaluators, mentors, developers, testers, and students who need to understand how the Sports_GenAi application is organized and how each module participates in article generation.

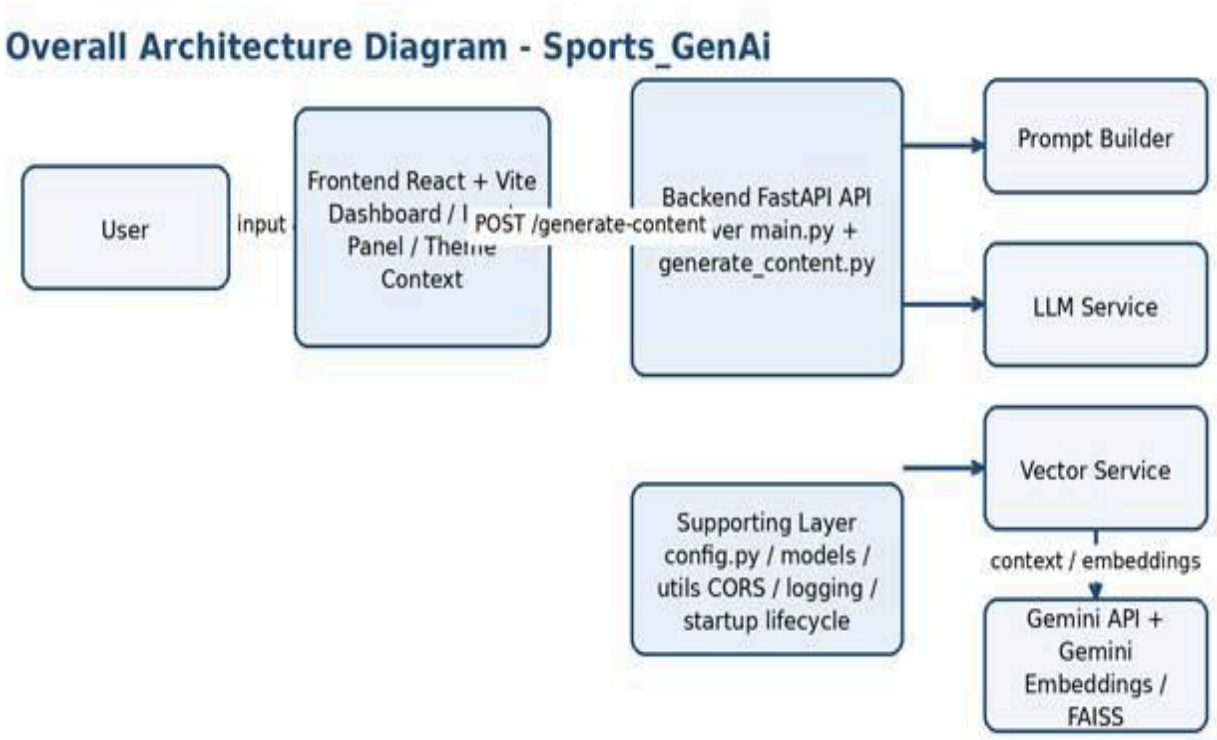
1.3 System overview

Sports_GenAi is a web-based generative AI application that creates sports articles from structured user input. The user provides the sport, content type, match title, highlights, tone, and desired length on the React dashboard. The FastAPI backend receives the request, retrieves related sports context from a FAISS vector store, and then sends a composed prompt to the LLM service. The generated article is returned as JSON and displayed on the frontend.

2. System Design

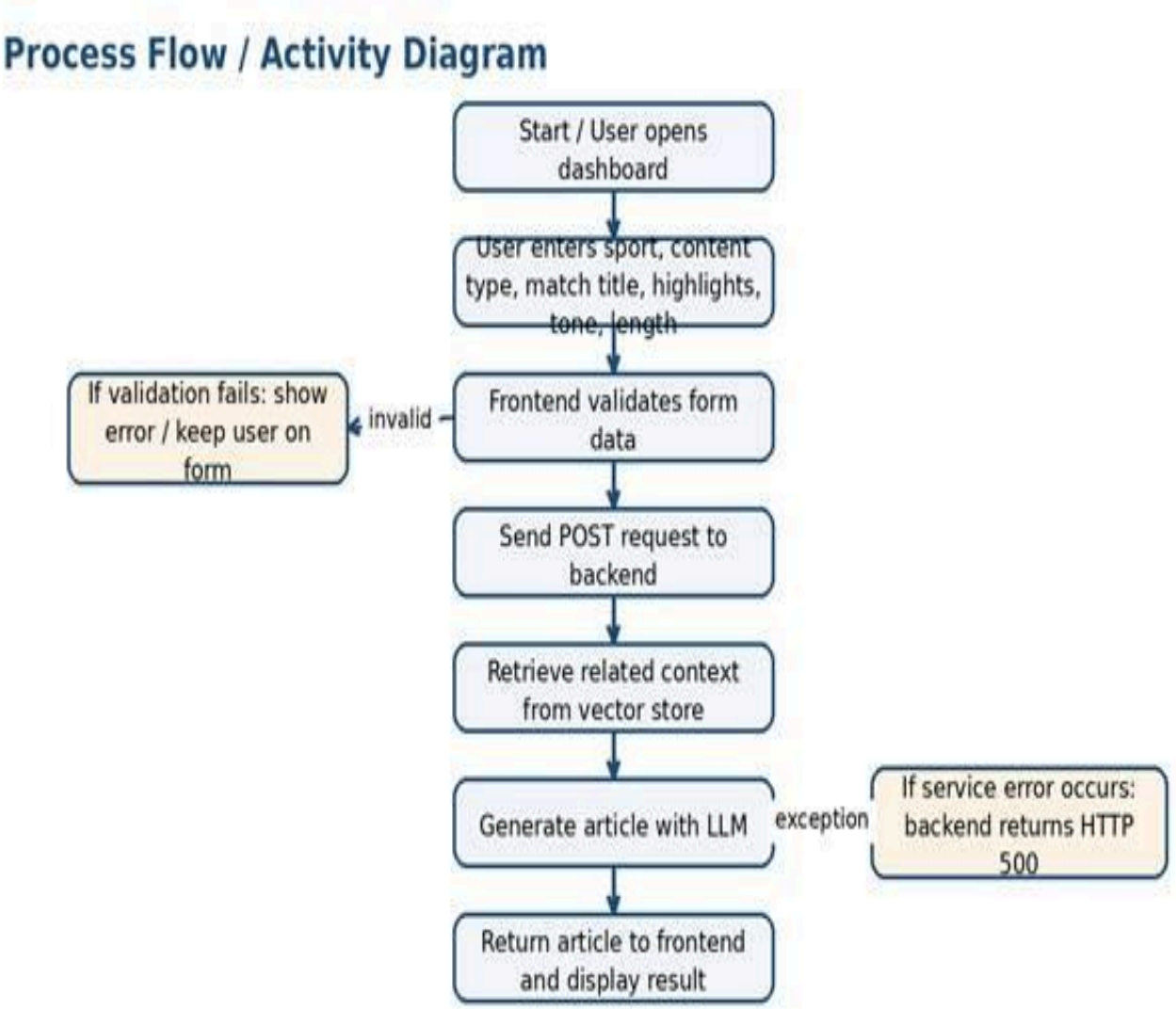
2.1 Application Design

The application follows a layered design. The frontend layer is built using React and Vite and includes components such as Dashboard.jsx and ThemeContext.jsx. The backend layer is built using FastAPI with a dedicated route for content generation. The service layer contains prompt_builder.py, llm_service.py, and vector_service.py. Supporting files such as config.py, models, and utils manage configuration and shared utilities.



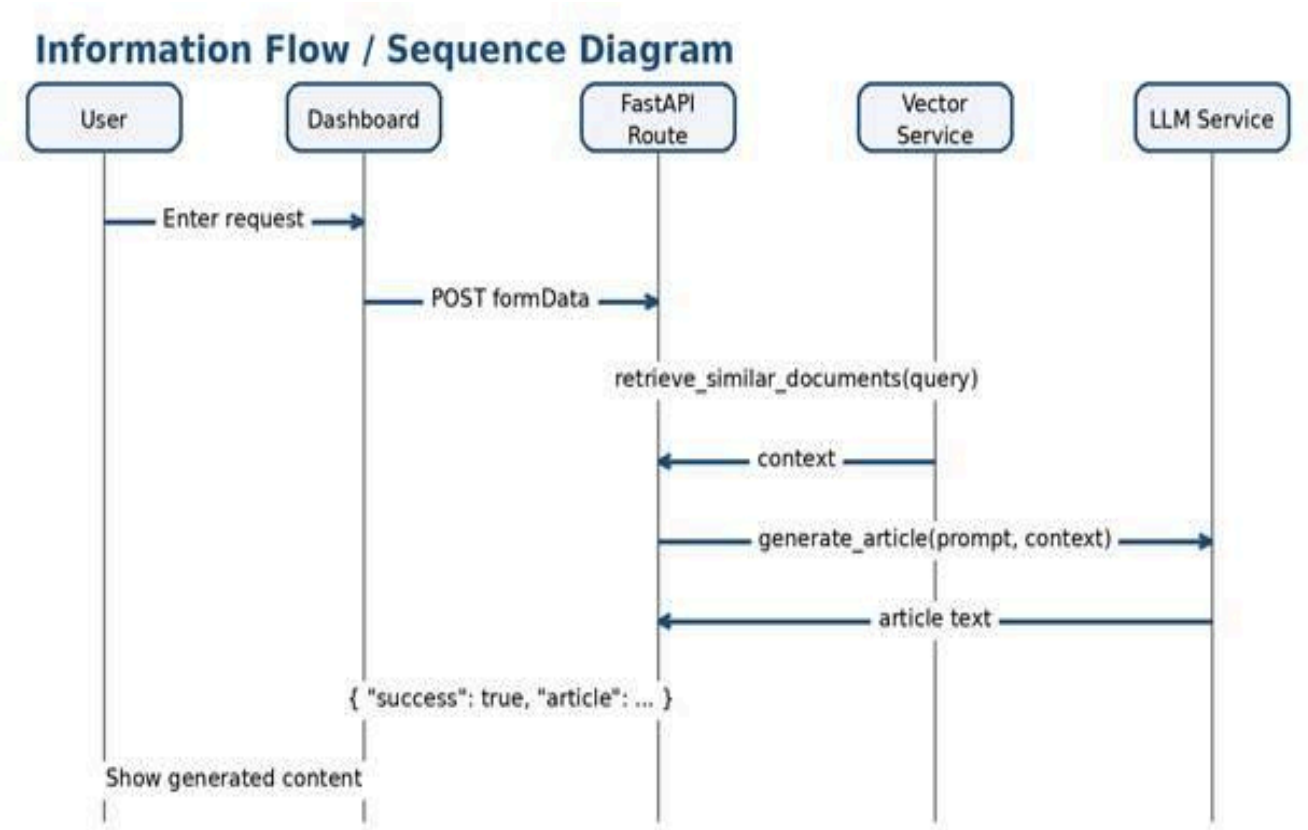
2.2 Process Flow

The process starts when the user opens the dashboard and enters content parameters. After client-side validation, the frontend sends a POST request to the backend. The backend retrieves relevant sports context from the vector database, builds the generation instruction, sends the final prompt to the LLM, and returns the generated article to the interface.



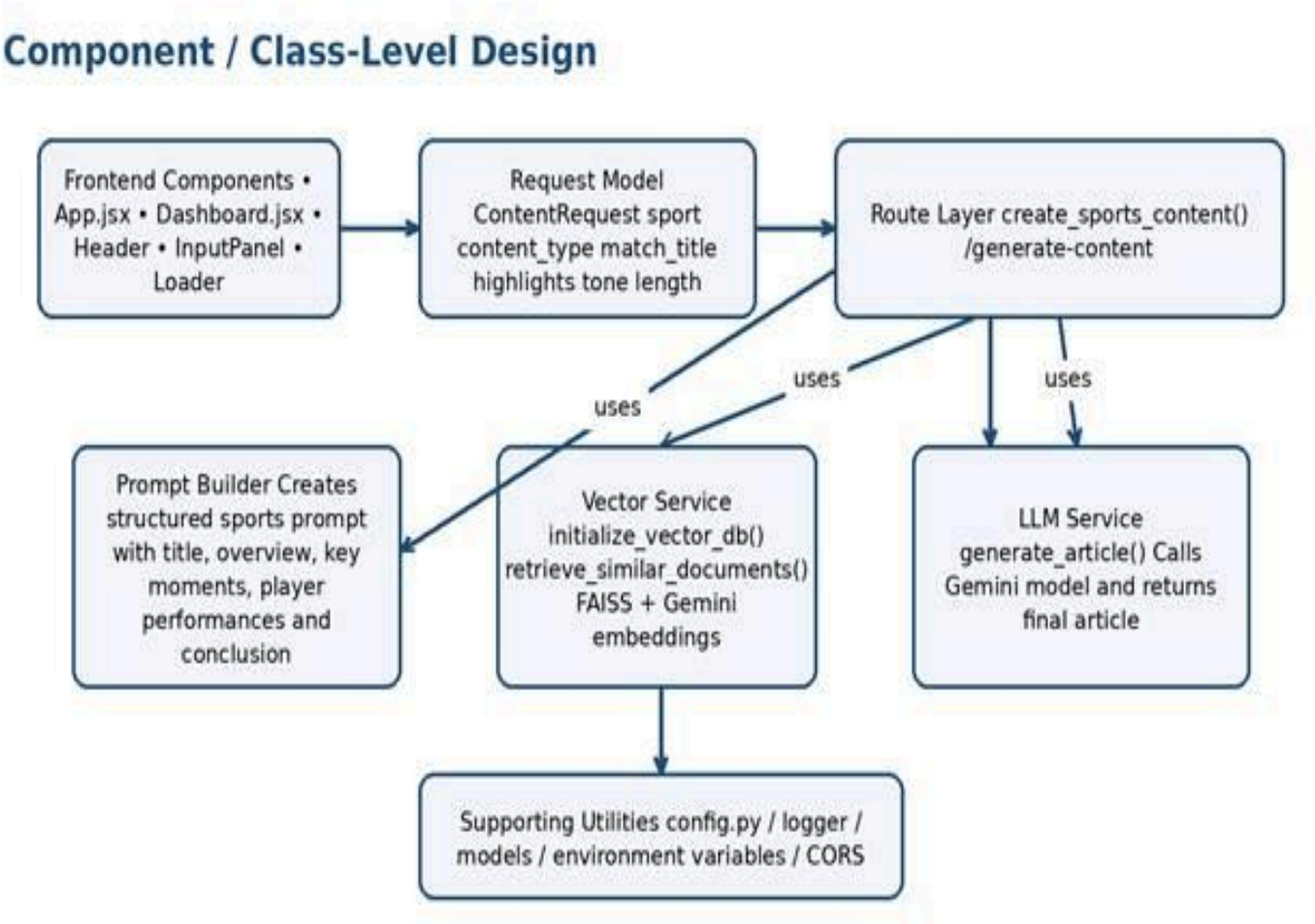
2.3 Information Flow

Information moves through the system in a sequential manner. User-entered form data is sent from the Dashboard to the FastAPI route. The route calls the vector service to fetch a similar context and then calls the LLM service to generate the final text. The backend returns a JSON response containing the article, which is then rendered in the frontend.



2.4 Components Design

The major components are shown below. The frontend collects inputs and shows the result. The request model defines the content request schema. The route layer orchestrates the generation call. The prompt builder converts user data into a structured sports-journalism prompt. The vector service manages embeddings and similarity search using FAISS and Gemini embeddings. The LLM service generates the final article.



2.5 Key Design Considerations

- • Separation of concerns: frontend, route, service, and support layers are kept logically separate.
- • Scalability: the use of a service layer allows the LLM and vector logic to be changed independently from the UI.
- • Prompt quality: the prompt builder follows a structured journalistic format including title, overview, key moments, player performances, turning points, and conclusion.
- • Context enhancement: vector similarity search provides domain-specific background information before generation.
- • Usability: the dashboard provides a simple form-based input workflow and shows loading state while the article is being created.
- • Reliability: the backend uses try/except blocks and HTTP 500 error handling for failures.
- • Cross-origin access: CORS middleware is configured to allow the local React application to communicate with the API.

2.6 API Catalogue

The main API endpoints used by the project are summarized below.

Endpoint	Method	Input	Output	Purpose
/generate-content	POST	sport, content_type, match_title, highlights, tone, length	{success, article}	Generate sports content
/health	GET	None	{"status": "running"}	Health check of backend

Known data model:

Field	Description
sport	Selected sport such as cricket or football
content_type	Type of output such as article, summary, report, or blog
match_title	Match or event title
highlights	Important points or match highlights
tone	Writing style, for example professional or exciting
length	Requested article size, such as short or detailed

3. References

- • Uploaded LLD template document used for report structure.
- • GitHub repository structure for Sports_GenAi.
- • backend/app/routes/generate_content.py
- • backend/app/services/prompt_builder.py
- • backend/app/services/vector_service.py
- • backend/app/config.py
- • backend/app/main.py
- • frontend/src/pages/Dashboard.jsx